# Práctica Final – Mario en Web

Diseño y Desarrollo de Videojuegos y Entornos Virtuales

Pol Gómez Morales

Promoción: 2027

Fecha de entrega: 29/03/26

UNIVERSIDAD DE DISEÑO,
INNOVACIÓN Y TECNOLOGÍA

## Indice:

**MEMORIA DEL PROYECTO.....1**
Desarrollo del nivel 1-1 de Super Mario Bros en HTML5 + JavaScript.....1
1. Introducción.....1
2. Arquitectura general del proyecto.....1
3. Motor de entidades.....1
Clase Entity.....1
4. Sistema de renderizado.....2
4.1 Renderizado de Entities.....2
4.2 Renderizado de Mario (y goombas).....2
5. Sistema de input.....2
6. Físicas: movimiento y salto.....3
7. Procesamiento por frame: cómo funciona el motor.....3
7.1. Cálculo de deltaTime.....3
7.2. Update de entidades.....3
7.3. Colisiones fase 1.....3
7.4. Colisiones fase 2 (resolución real).....3
7.5. Limpieza.....4
7.6. Movimiento de cámara.....4
7.7. Renderizado.....4
8. Sistema de colisiones.....4
8.1 Detección: AABB.....4
8.2 Resolución.....4
8.3 Interacciones especiales.....4
8.4 Active band.....4
9. Cámara lateral.....5
10. Sistema de niveles.....5
11. Entidades especiales del nivel.....5
Question Block.....5
Brick.....5
Goomba.....5
Mushrooms.....6
Bandera.....6
12. Mario: estados y comportamiento.....6
13. Sistema de audio.....6
14. Interfaz (HUD).....6
Hud Principal:.....6
Menu de Pausa:.....7
15. Optimización.....7
16. Problemas y soluciones.....7
1. Colisiones.....7
2. Salto.....7
17. Conclusión.....7

# MEMORIA DEL PROYECTO

## Desarrollo del nivel 1-1 de *Super Mario Bros* en HTML5 + JavaScript

## 1. Introducción

Este proyecto consiste en la creación de una recreación funcional del nivel **1-1** de *Super Mario Bros* utilizando únicamente **HTML5 Canvas** y **JavaScript**, sin engines externos.
El objetivo era demostrar la capacidad de:

- Programar un pequeño motor de juego.
- Gestionar renderizado, animaciones y físicas básicas.
- Implementar interacción entre entidades: enemigos, power-ups, bloques, monedas…
- Cargar un nivel completo a partir de datos externos.

El trabajo se realizó desde cero, organizando el código por módulos y manteniendo una arquitectura sencilla pero escalable.

Link al GitHub: https://github.com/Dagger-PolGomez/Mario-In-Web
Link a la pagina: https://dagger-polgomez.github.io/Mario-In-Web/

## 2. Arquitectura general del proyecto

El juego está dividido en carpetas según su función:

- **Entidades**: Mario, Goomba, Mushroom, bloques, tuberías, escaleras, bandera, etc.
- **/engine**: sistema de colisiones.
- **/levels**: mapas y cargador de niveles.
- **/audio**: gestor de música y efectos.
- **/utils**: constantes y valores globales.
- **game.js**: game loop, cámara, procesamiento de cada frame.
- **index.html**: inicialización del canvas.

Esta estructura permite separar claramente la lógica del juego, el contenido del nivel y los componentes reutilizables.

## 3. Motor de entidades

Todas las entidades del juego heredan de una clase base:

### Clase Entity

Incluye los atributos esenciales:

- **x, y**: posición en el mundo.

- **width, height**: tamaño lógico del sprite.
- **vx, vy**: velocidad.
- **onGround**: si está apoyado en el suelo.
- **isSolid**: si bloquea a otras entidades.
- **remove**: marcar para eliminar.
- **update()** y **render()**: funciones comunes.

Gracias a esto, todas las entidades se comportan de forma coherente dentro del motor.

# 4. Sistema de renderizado

El renderizado se realiza con **Canvas 2D**, dibujando frame a frame:

1. Se limpia la pantalla y se pinta un cielo azul.
2. Se aplica una **transformación** al contexto (`ctx.translate`) para desplazar todo según la cámara.
3. Se recorren todas las entidades y se les llama a `render(ctx,img)`.
4. Cada entidad dibuja su porción del spritesheet.

El spritesheet completo del juego está en una imagen de 160×160 por tile, escalados por **RENDER_SCALE** para verse nítidos.

## 4.1 Renderizado de Entities

Cada entidad se renderiza en base a la SpriteSheet referencia. Además todas las entidades tienen dos variables “SSx” y “SSy” que son la X e Y de la Spritesheet donde empiezan. Cuando el motor llama la función de renderizar, la entidad utiliza el SSx y SSy para encontrar el sprite.

Como todas los sprites (menos Mario) tienen el mismo tamaño, no hace falta especificar y se puede hardcodear.

## 4.2 Renderizado de Mario (y goombas)

Mario sigue una lógica muy parecida a las otras entidades, pero tiene algunos mas parámetros. Estos son:

- Booleano para cuando salta
- Integer para cuanto de grande es
- Listas de las X e Y de las diferentes animaciones
- Lista de integers de cuantas frames tienen su animación respectiva

Utilizando eso parametros, renderizamos una sprite u otra

Los goombas son lo mismo, pero sin la parte de saltar o cuanto de grande son.

# 5. Sistema de input

Se controla con listeners de teclado.
El jugador puede:

- Moverse a izquierda y derecha.

- Saltar manteniendo o pulsando brevemente la tecla.

El input solo modifica velocidades; las físicas y colisiones deciden el movimiento final.

# 6. Físicas: movimiento y salto

Mario y los enemigos funcionan con un sistema sencillo de físicas 2D:

- **Gravedad constante** que sumamos cada frame.
- **Velocidad horizontal estable** dependiente del input o del tipo de entidad.
- **Salto progresivo**:
    - Con un impulso inicial (JUMP_FORCE).
    - Con un extra mientras se mantiene pulsado (JUMP_HOLD y MAX_JUMP_TIME).

Esto permite reproducir la sensación del salto clásico de Mario.

# 7. Procesamiento por frame: cómo funciona el motor

El juego funciona con un **game loop** que se ejecuta unas 60 veces por segundo.
En cada frame ocurre lo siguiente:

## 7.1. Cálculo de `deltaTime`

Tiempo desde el frame anterior, para que el juego sea estable aunque el FPS varíe.

## 7.2. Update de entidades

- Mario se actualiza siempre.
- Goombas y Mushrooms solo se actualizan si están dentro de un "**active band**" alrededor de la cámara (optimización).
- Cada entidad actualiza su propia lógica interna (movimiento, físicas o animaciones).

## 7.3. Colisiones fase 1

Se clasifican las entidades en:

- **Solids** (tuberías, bloques, escaleras, bandera…)
- **Movers** (Goombas y Mushrooms)
- **Mario**

Por razones de optimización, aquí también se gestionan interacciones no sólidas:

- Mario recoge monedas.
- Mario coge una seta.
- Mario aplasta un Goomba.
- Mario toca la bandera.

## 7.4. Colisiones fase 2 (resolución real)

Se resuelven colisiones físicas contra cualquier cosa sólida:

- **Mario vs sólidos**
- **Goombas/Mushrooms vs sólidos**

El motor decide:

- De qué lado ocurrió la colisión.
- Si un personaje aterriza, choca contra una pared o golpea desde abajo.
- Cambio de dirección de enemigos.
- Rotura de bloques si Mario es grande.

### 7.5. Limpieza
Entidades marcadas para borrar se eliminan.

### 7.6. Movimiento de cámara
Solo avanza hacia la derecha, siguiendo a Mario.

### 7.7. Renderizado
Se dibuja todo en la pantalla, incluyendo la UI.

## 8. Sistema de colisiones
Este es uno de los apartados más técnicos del proyecto.

### 8.1 Detección: AABB
Todas las colisiones se detectan con **Axis-Aligned Bounding Box**:

```
Si (A.x < B.x + B.width) &&
   (A.x + A.width > B.x) &&
   (A.y < B.y + B.height) &&
   (A.y + A.height > B.y)
     → hay colisión.
```

Es rápido y perfecto para juegos 2D retro.

### 8.2 Resolución
Cuando hay colisión, se calcula:
- Distancia de penetración en X
- Distancia de penetración en Y

Se resuelve por el eje donde la penetración es menor.
Ejemplos:
- Si Mario cae y la penetración vertical es menor → se considera que **aterriza**.
- Si Mario choca lateralmente → se **bloquea la pared** y no puede avanzar.
- Si golpea un bloque desde abajo → se ejecuta la lógica de ese bloque.

### 8.3 Interacciones especiales
- **Goomba stomp**: Mario debe venir con velocidad vertical positiva (cayendo).
- **Daño a Mario**: si la colisión no es por arriba, se resta tamaño o muere.
- **Seta**: interacción no sólida; si colisiona, Mario crece y la seta se borra.

### 8.4 Active band
Para evitar actualizar centenares de Goombas lejos de la cámara:

- Se calcula una banda alrededor del jugador ≈ 1.5× cámara.

- Solo dentro de esa banda se actualizan enemigos.

Esto mejora mucho el rendimiento.

# 9. Cámara lateral

La cámara sigue a Mario solo hacia adelante:

- Si Mario supera la mitad de la pantalla → la cámara avanza.
- No se permite retroceder más allá del borde izquierdo.
- No existe scroll vertical.

Da el comportamiento clásico del Mario de NES.

# 10. Sistema de niveles

El nivel está definido como una **matriz gigante de símbolos** (level1_1.js).
Cada símbolo representa una entidad:

```
m = Mario
b = Brick
f = suelo
q = Question Block
qm = Question Block con seta
g = Goomba
pLT, pRT, pL, pR = tuberías (partes izquierda/derecha)
s = escalones
p = bandera
c = moneda
```

El loader recorre la matriz y crea las entidades según corresponda, respetando posiciones en mundo.

# 11. Entidades especiales del nivel

## Question Block

- Si no está usado:
    - Da una moneda o genera una seta emergiendo.
- Se marca como “used” para evitar más activaciones.

## Brick

- Si Mario es grande: se rompe.
- Si es pequeño. solo hace “bump”.

## Goomba

- Se mueve continuamente en una dirección.

- Cambia de dirección al chocar con una pared.
- Puede ser aplastado.

### Mushrooms

- Primero emergen del bloque.
- Después se mueven horizontalmente.
- Se chocan con bloques igual que los enemigos.

### Bandera

Tocar la bandera dispara:

- Congelar a Mario.
- Música de victoria.
- Fin del nivel tras unos segundos.

## 12. Mario: estados y comportamiento

Mario soporta diferentes estados:

- **Tamaño pequeño / grande**
- **Invencible durante daño** (parpadea)
- **Saltando**
- **Muerto** (animación + recarga del nivel)
- **Movimiento bloqueado** durante la cinemática final

Cada animación depende de la velocidad y el estado actual.

## 13. Sistema de audio

El AudioManager gestiona:

- Música de fondo (overworld).
- Sonidos: salto, stomp, moneda, power-up, bloque, victoria…

La música solo empieza cuando el usuario interactúa (requisito de navegadores modernos).

## 14. Interfaz (HUD)

### Hud Principal:

En la parte superior:

- MARIO y score
- COINS
- LIVES
- TIME

Se pinta encima del juego y no se afecta por la cámara.
El sistema es simple aproposito para claridad, tomado como referencia el juego original de Mario

### Menu de Pausa:

El sistema de pausa se añadió para permitir detener el juego temporalmente sin cerrar la página ni reiniciar el nivel. Para ello se creó un estado de pausa dentro del juego que se activa o desactiva al presionar la tecla **Escape**. Cuando el juego está en pausa, la lógica del juego deja de actualizarse, por lo que los personajes, enemigos y objetos quedan completamente congelados, aunque la escena sigue mostrándose en pantalla.

Además, al activarse la pausa se dibuja una capa semitransparente sobre el juego junto con un pequeño menú que muestra el texto **"PAUSE"** y un botón para reanudar la partida. Este botón cambia ligeramente de color cuando el cursor pasa por encima para indicar que es interactivo, y al hacer clic permite continuar el juego. De esta manera, el jugador puede pausar y reanudar la partida tanto con la tecla **Escape** como utilizando el botón del menú de pausa.

### Menu Principal:

Se ha diseñado una pantalla inicial renderizada directamente en el canvas, utilizando un fondo con colores diferenciados para simular profundidad, un título principal destacado con efecto de sombra y una ligera animación vertical que le aporta dinamismo. Además, se ha añadido un subtítulo que guía al usuario, indicando cómo comenzar la partida.

El botón de inicio también ha sido optimizado para mejorar la interacción. Se ha implementado un sistema de detección del cursor que permite identificar cuándo el usuario pasa el ratón sobre el botón, aplicando un cambio de color y un ligero aumento de tamaño para simular un efecto de *hover*. Asimismo, se ha añadido una sombra para dar sensación de relieve. Estas mejoras no solo hacen el menú más visualmente atractivo, sino que también refuerzan la experiencia de usuario al proporcionar una interfaz más clara y responsiva.

## 15. Optimización

El proyecto incluye varias mejoras:

- **Active band** para enemigos fuera de cámara.
- **Limpieza automática** de entidades caídas fuera del mapa.
- **Uso de un único spritesheet** para todo.
- **Minimización de cálculos por frame** (no recalcular listas innecesariamente).

## 16. Problemas y soluciones

Durante el desarrollo surgieron varios retos:

### 1. Colisiones

Las colisiones fue lo que mas me duro de tiempo de ajustar y hacer bien.
Inicialmente si hizo un sistema de AABB, pero el problema era que algunas colisiones (y especialmente de los enemigos) funcionaban mal.

Al final se decidio de hacer un sistema mas complejo mejorado de AABB, en el que todas las entidades mantienen su ultima posición, y el resultado de la colisión era un diccionario con información de que lado estaba colisionando (punto de vista del que se mueve), y qué lado venía la colisión (punto de vista del que es estático

### 2. Salto

El salto se sentía muy poco controlando durante el desarrollo porque no había funcionalidad de mantener el botón y hacer que Mario saltara más de lo normal. Esto se implementó añadiendo fuerza extra si se estaba presionando la tecla.

## 17. Conclusión

Este proyecto reproduce fielmente el comportamiento fundamental del Mario clásico:

- Movimiento fluido
- Saltos controlables
- Enemigos básicos
- Power-ups
- Sistema de bloques
- Colisiones sólidas
- Renderizado constante

La estructura modular facilita añadir nuevos niveles o personajes en el futuro.
Como mejoras futuras:

- Implementar Koopas y plataformas móviles
- Añadir animaciones más complejas
- Añadir un menú principal
- Implementar scroll vertical
- Crear más niveles del juego original

La razón porque se eligió hacerlo de esta forma y con un “loader” fue porque este proyecto lo hice hace 4 años en Python con otra librería, pero tuve que hacer el mismo ejercicio de generar el motor y el renderizador. Durante ese proyecto fue donde me di cuenta que seria muy util tener un formato en el que pueda añadir o quitar elementos de una forma modular, y un array (aunque ineficiente para cargar la pagina) es la forma más cómoda de hacerlo

La única diferencia en términos de programación ha sido la parte de hacer rendering porque tenía que adaptarme a como javascript procesa los datos, pero aun así no tuvo mucha más complejidad.

En términos de mejoras, se podría mejorar aún más las colisiones, porque hay un pequeño bug que mario no puede saltar si su movimiento es en diagonal, y está en el suelo, haciendo que mario no pueda saltar. Y además se podría añadir la persistencia de vidas de mario.

El resultado final demuestra un buen entendimiento de los fundamentos de un motor 2D, del diseño basado en entidades y cómo traducirlas a un renderer customizado en web.